Memory in Toy Transformer Models

This document is not finished, many sections are missing or very incomplete. Some sections are drafted by Claude and I have not yet verified that everything is correct. In conclusion, do not trust this document in its current form.

1 Introduction

The goal of this project is to understand how memorised facts are stored in the weights of transformer models. To isolate the mechanisms involved, we study a minimal *toy model*: a single-layer transformer that receives two input tokens and must produce one output token. A *fact* is defined as a specific pair of input tokens mapped to a specific output token; the mapping is random, so the model cannot exploit any systematic structure and must genuinely memorise each fact.

By selectively enabling or disabling components of the architecture — attention, feed-forward layers, biases, normalisation, residual connections — we first find out what parts of the model is important for fact memorisation. After this, we select a small number of model configurations, that we scale up to see how the maximum possible number of memorised facts, scale with model size.

In addition, we attempt to match the trained model's performance with a hand-coded weight-setting algorithm (i.e. not learned via gradient descent). Why attempt this? Because a good measure of if you understand something, is if you can replicate it. I.e. if we understand how the model is storing facts in its weights, we should be able to hand-code a model that does the same thing.

Our attempt to match the trained model's performance does not succeed, but we do think our approach is some non-zero amount of progress.

2 Model Architecture

The model, MemoryToyModel, is a single-layer transformer whose components can be toggled on or off via a set of architectural flags. All configurable options are collected in a ModelSettings object; the full list is summarised in Table 1 and described below.

2.1 Overview

The forward pass proceeds through up to six stages:

1. **Embedding**: The input tokens are mapped into a d_{residual}-dimensional residual stream. The precise mechanism depends on whether attention is enabled. (See Section 2.2 for details.)
2. **Attention** (optional): If enabled, the embedded sequence is processed by a single self-attention layer. After the attention, only the last sequence position is retained. (See

Section 2.3 for details.)

3. **Layer Norm 1** (optional): An RMSNorm layer normalises the residual stream. This layer norm is present if and only if both attention and norms are enabled.
4. **Feed-Forward block** (optional): If enabled, the activations are passed through either a GELU or ReLU block, with a residual connection around this block as an optional extra. (See Section 2.4 for details).
5. **Layer Norm 2** (optional): An RMSNorm layer normalises the residual stream. This layer norm is present if and only if both feed-forward and norms are enabled.
6. **Output Head**: A final linear projection maps the residual stream to logits over the output vocabulary.

Parameter	Values	Description
Data dimensions		
`seq_len`	2	Number of input tokens per fact
`input_vocab_size`	32, 64, 128, 256	Size of the input token vocabulary
`output_vocab_size`	16, 32, 64, 128	Size of the output token vocabulary
`n_facts`	int	Number of facts to memorise
`seed`	42	Random seed for fact generation
Internal dimensions		
`d_residual`	16, 32, 64, 128	Residual-stream (embedding) dimension
`n_heads`	1	Number of attention heads
`d_ff`	16, 32, 64, 128	Hidden dimension of the feed-forward block
Architectural toggles		
`attention`	True/False	Enable causal self-attention
`ff`	True/False	Enable feed-forward block
`bias`	True/False	Include bias terms in linear layers
`norms`	True/False	Use RMSNorm (otherwise identity)
`ff_residual`	True/False	Residual connection around FF block
`ff_activation_type`	GELU/ReLU	Activation function (GELU or ReLU)

Table 1: All possible settings for MemoryToyModel, and the different values used in experiments. The code allows the *Data dimensions* and *Internal dimensions* (including `seq_len`,`n_heads` and `seed`) to be set to different values than those listed, but the ones listed are the only ones actually used. Feel free to experiment yourself with other values.

2.2 Embedding

Two modes are available, controlled by the `attention` flag.

With attention (`attention = True`). Each input token is embedded via a learned token embedding $W_E \in \mathbb{R}^{V_{\text{in}} \times d_{\text{residual}}}$ and summed with a learned positional embedding $W_P \in \mathbb{R}^{T \times d_{\text{residual}}}$, where T is the sequence length and V_{in} is the input vocabulary size. The full embedded sequence is normalised with RMSNorm (if `norms = True`, otherwise the identity) and passed through a causal self-attention layer with n_{heads} heads (Section 2.3). Only the representation at the *last* sequence position (T) is retained, combined with the embedding at the same position via a residual connection that is always active (not controlled by any flag):

$$x = e_T + \text{Attn}(\text{Norm}(e))_T \,,$$

where $e \in \mathbb{R}^{T \times d_{\text{residual}}}$ is the embedded sequence and subscript T denotes the last position. Note that the attention operates on the *full* sequence (all positions attend to their causal context), but only the output at position T is used going forward.

Without attention (`attention = False`). A separate embedding matrix $W_{E_i} \in \mathbb{R}^{V_{\text{in}} \times d_{\text{residual}}}$ is learned for each input position $i \in \{1, \ldots, T\}$. The residual-stream vector is the element-wise sum of all position embeddings:

$$x = \sum_{i=1}^{T} W_{E_i}[\text{token}_i].$$

This removes the attention mechanism entirely, replacing it with a simple additive composition of per-position embeddings.

2.3 Attention

When attention is enabled, the model uses multi-head causal (masked) self-attention. A single linear projection produces queries, keys, and values (Q, K, V), which are split across n_{heads} heads, each of dimension $d_{\text{head}} = d_{\text{residual}}/n_{\text{heads}}$. Attention scores are computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}} + M\right) V,$$

where M is a causal mask that sets future positions to $-\infty$. The outputs of all heads are concatenated and projected back to d_{residual} dimensions via a learned linear map. Notably, no bias is used in the attention projections.

2.4 Feed-Forward Block

When enabled (`ff = True`), the feed-forward block is a standard two-layer MLP:

$$\text{FF}(x) = W_2\, \sigma(W_1 x + b_1) + b_2,$$

where $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{residual}}}$, $W_2 \in \mathbb{R}^{d_{\text{residual}} \times d_{\text{ff}}}$, and σ is a configurable activation function (GELU by default; ReLU is also supported). Biases b_1, b_2 are included only when `bias = True`. The input to the MLP is optionally normalised with RMSNorm (controlled by `norms`).

A residual connection around the feed-forward block is controlled by the flag `ff_residual`:

$$x \leftarrow \begin{cases} x + \text{FF}(\text{Norm}(x)) & \text{if } \texttt{ff\_residual = True}, \\ \text{FF}(\text{Norm}(x)) & \text{otherwise.} \end{cases}$$

2.5 Output Head

The residual-stream vector is passed through a final RMSNorm (if `norms = True`) and then a linear projection $W_H \in \mathbb{R}^{V_{\text{out}} \times d_{\text{residual}}}$ (with bias when `bias = True`) to produce logits over the output vocabulary of size V_{out}. This final norm is always present regardless of whether attention or the feed-forward block is enabled.

3 Fact Generation

Facts are generated deterministically from the random seed. Each fact consists of `seq_len` input tokens drawn (without replacement at the tuple level) from the input vocabulary and a target token assigned cyclically from the output vocabulary ($\text{target}_i = i \bmod V_{\text{out}}$). This ensures a uniform distribution over output classes while the input–output mapping remains arbitrary, preventing the model from learning a simple rule.

4 Training

The model is trained with the Adam optimiser, using binary cross-entropy with logits as the loss function (applied to one-hot encoded targets). Two accuracy metrics are tracked:

- **Accuracy**: the fraction of (fact, output-class) pairs for which the sign of the logit matches the one-hot label.
- **Best-guess accuracy**: the fraction of facts for which arg max of the logits equals the correct target.

Training supports optional early stopping: it halts when perfect monitored accuracy is reached, or when the monitored accuracy has not improved for a specified number of patience epochs.

Parameter	Supported by code	Used in experiments	Description
Training parameters			
`n_epochs`	int	50 000	Maximum training epochs
`lr`	float	1e-2, 3e-3, 1e-3	Learning rate
`optimizer_type`	Adam, AdamW	Adam	Optimiser
`grad_clip_norm`	float or None	None	Gradient clipping norm
`smoothing`	float or None	None	Label smoothing factor
`early_stopping`	True/False	True	Enable early stopping
`patience`	int	3 000–5 000	Epochs without improvement before stopping
`target_accuracy`	`accuracy`, `best_guess_accuracy`	`accuracy`	Monitored metric for early stopping
`loss_type`	BCE, CE	BCE, CE	Loss function type (Binary Cross-Entropy or Cross-Entropy)
Capacity search parameters			
`lr`	list of floats	[1e-2, 3e-3, 1e-3]	Learning rates, used sequentially. When progress has stalled or max epochs reached, continue with the next learning rate.
`precision`	int	8	Binary search stops when the remaining search interval is smaller than this value.
`number_of_attempts`	int	1, 3	Number of times to try to learn each number of facts, re-starting with a new randomly initialised model each time.
`threshold_to_continue`	float	0.99, 0.95	If the monitored accuracy is below this threshold, when it's time to move on to the next learning rate, just give up instead.

Table 2: Training and capacity-search parameters. "Supported by code" lists the types or values accepted by `train_model` and `find_max_facts`; "Used in experiments" lists the values actually used in the recorded runs.

5 Experiment: Which architectural components matter for fact memorisation?

The results of this experiment are shown in Table 5, which reports the maximum number of facts successfully memorised for each architectural configuration. Outliers are highlighted with a box and the highest value in each row is shown in **bold**.

The results comes from four diffrent runs: `experiment_log_3att_duplicate`, `experiment_log_3att`,

File	patience	target_accuracy	loss_type	precision	number_of_attempts	threshold_to_continue
E1/experiment_log.jsonl	5000	?	BCE	8	1	0
E2/experiment_log.jsonl	5000	?	BCE	8	1	0.99
E2/experiment_log_3att.jsonl	3000	?	BCE	8	3	0.99
E2/experiment_log_3att_duplicate.jsonl	3000	?	BCE	8	3	0.99
E2/experiment_log_CE_3att.jsonl	3000	?	CE	8	3	0.95
E2/experiment_log_CE_3att_duplicate.jsonl	3000	?	CE	8	3	0.95
E3/full_model_log.jsonl	5000	?	BCE	[8, 32, 128, 512]	3	0.99
E3/full_model_log_duplicate.jsonl	5000	?	BCE	[8, 32, 128, 512]	3	0.99
E3/reduced_model_log.jsonl	5000	?	BCE	[8, 32, 128, 512]	3	0.99
E3/reduced_model_log_duplicate.jsonl	5000	?	BCE	[8, 32, 128, 512]	3	0.99

Table 3: Per-file training and capacity-search settings for the experiment logs in `E1`, `E2`, and `E3`. Entries are left as placeholders for manual completion.

`experiment_log`, and `E1/experiment_log`. For the last one only results with attentions are included becasue of a bugg in the code for non-attention cases, during that run.

5.1 Comments on outliers

Outliers are highlighted with a box. Almost all outliers are low values, which indicates that they are due to training failures. The one exception is the high outlier of 768 facts memorised for the configuration with Attention, FF, Norms, Residual, Bias, and ReLU activation. I don't now what's up with that, but I think it indicates that higher number of facts is generaly possible, whith a bit of luck, and/or longer training.

There are lots of outliers in the experiments which did not run mulitple attempts for each number of facts. This shows that running multiple attempts is important for avoiding being misled by training failures.

5.2 Attention and FF obviously matters

Including Attention or not, and incluing a Feed-Forward block or not, obviouly matters, which is also confirmed in table 5.

Notice that excluding Attention leads to more facts memorised. This is not supprising. The reason is beause the attention replacement is more powerfull than the attention itself.

Next we'll go over the other settings one by one, in order of least expected importance.

5.3 Does GELU vs RELU matter? - Probably not much

I don't think GELU vs ReLU has a significant impact on the maximum number of facts that can be learned. The main reason for this conclusion is that niether GELU nor RELU consistently outperform the other option, across the different architectural configurations.

Data and model dimension parameters	Value
`input_vocab_size`	32
`output_vocab_size`	16
`d_residual`	16
`d_ff`	16

Table 4: Fixed settings for the experiment in Section 5. The architectural toggles (`attention`, `ff`, `bias`, `norms`, `ff_residual`, and `ff_activation_type`) are varied across runs, as shown in Table 5.

Looking at table 5, we can see that the choice of activation function (GELU vs ReLU) does not have a consistent impact on the maximum number of facts memorised across different architectural configurations. Comparing GELU vs ReLU for each combination of the otther settings, we find that **GELU have higher Max Facts in 9 cases, ReLU have higher Max Facts in 5 cases, and they score equaly in 2 cases.**

Most of the time the diffrence in Max Facts is small, except for

- Attention: ✓, FF: ✓, Norms: ✓, Residual: ✓, Bias: ✓
  where the difference is 576 vs 768, in favour of ReLU.
- Attention: ✓, FF: ✓, Norms: ✓, Residual: ✗, Bias: ✗
  where the difference is 528 vs 384 in favour of GELU.
- Attention: ✓, FF: ✓, Norms: ✗, Residual: ✓, Bias: ✓
  where the difference is 304 vs 432 in favour of ReLU.

I think these are due to training instabilities rather than a significant fact learning capacity between ReLU and GELU.

5.4 Does including a Bias matter - Probably not much

Including Bias or not in the feed-forward block does not seem to have a bit more impact than the choice of activation function. Bias mostly does better than No Bias, but there are some counter examples.

Looking at table 5, we can see that the inclusion of a Bias in the FF module, does not have a consistent impact on the maximum number of facts memorised across different architectural configurations. Comparing Bias vs No Bias for each combination of the otther settings, we find that **Bias have higher Max Facts in 11 cases, No Bias have higher Max Facts in 3 cases, and they score equaly in 1 case.**

The largest differences in Max Facts between Bias and No Bias are observed in the following configurations:

- Attention: ✓, FF: ✓, Norms: ✓, Residual: ✓, Activation: ReLU
  where the difference is 768 vs 536 in favour of Bias.
- Attention: ✗, FF: ✓, Norms: ✓, Residual: ✗, Activation: GELU
  where the difference is 792 vs 584 in favour of Bias.
- Attention: ✗, FF: ✓, Norms: ✓, Residual: ✗, Activation: ReLU
  where the difference is 704 vs 512 in favour of Bias.

There are some effects of including the Bias. The effect size is bigger than for GELU vs ReLU. But this effect should decreese as the network scales, so going forward, I will chose not to worry

about this.

5.5 Does including a Residual connection matter? - Yes

Looking at table 5, we can see that the inclusion of a Residual connection in parallel to the FF module, has a mostly consistent impact on the maximum number of facts memorised across different architectural configurations. Comparing Residual vs No Residual for each combination of the other settings, we find that **Residual have higher Max Facts in 15 cases, No Residual have higher Max Facts in 1 case.**

The one case where No Residual outperforms Residual is:

- Attention: ✓, FF: ✓, Norms: ✗, Bias: ✓, Activation: GELU
  where the difference is 308 vs 384 in favour of No Residual.

For the rest of the configurantions, Residual outperforms no Residual with 5.1% - 50.0% (25.9% average) more Facts memorised.

5.6 Does including a Layer Norms matter? - Yes

Looking at table 5, we can see that the inclusion of a Layer Norms, has a mostly consistent impact on the maximum number of facts memorised across different architectural configurations. Comparing Layer Norms vs No Layer Norms for each combination of the other settings, we find that **Layer Norms have higher Max Facts in 17 cases, No Layer Norms have higher Max Facts in 1 case.**

The one case where No Layer Norms outperforms Layer Norms is:

- Attention: ✗, FF: ✓, Norms: ✗, Bias: ✗, Activation: GELU
  where the difference is 512 vs 552 in favour of No Layer Norms.

The inclusion of Layer Norm has a much large inpact on when there is no FF module. Excluing the two no FF cases, and the configuration mentioned above, Layer Norms outperforms no Layer Norms with 4.7% - 89.5% (32.0% average) more Facts memorised.

When there is no FF module, Layer Norms outperforms no Layer Norms with 300.0% - 433.3% (366.7% average) more Facts memorised.

6 Experiment: How does Max Facts scale with model size?

Attention	FF	Norms	Residual	Bias	Activation	Max Facts				
						Dup	3att	Orig	E1	
✓	✓	✓	✓	✓	GELU	568	**576**	248	528	⋆
✓	✓	✓	✓	✓	ReLU	536	536	**768**	368	
✓	✓	✓	✓	✗	GELU	560	**576**	**576**	512	
✓	✓	✓	✓	✗	ReLU	**536**	504	496	512	
✓	✓	✓	✗	✓	GELU	504	504	464	**520**	
✓	✓	✓	✗	✓	ReLU	456	**512**	432	56	
✓	✓	✓	✗	✗	GELU	**528**	**528**	120	480	
✓	✓	✓	✗	✗	ReLU	368	**384**	288	320	
✓	✓	✗	✓	✓	GELU	**304**	264	48	80	
✓	✓	✗	✓	✓	ReLU	**432**	304	160	160	
✓	✓	✗	✓	✗	GELU	368	**424**	152	288	
✓	✓	✗	✓	✗	ReLU	416	**512**	96	120	
✓	✓	✗	✗	✓	GELU	**384**	232	304	256	
✓	✓	✗	✗	✓	ReLU	248	**368**	320	296	
✓	✓	✗	✗	✗	GELU	280	240	224	**336**	
✓	✓	✗	✗	✗	ReLU	320	272	**352**	8	
✓	✗	✓	N/A	N/A	N/A	**256**	**256**	200	24	
✓	✗	✗	N/A	N/A	N/A	**48**	**48**	24	32	
✗	✓	✓	✓	✓	GELU	**832**	792	808		
✗	✓	✓	✓	✓	ReLU	808	816	**832**		
✗	✓	✓	✓	✗	GELU	**744**	**744**	728		
✗	✓	✓	✓	✗	ReLU	**744**	736	728		
✗	✓	✓	✗	✓	GELU	**792**	768	768		
✗	✓	✓	✗	✓	ReLU	**704**	632	680		
✗	✓	✓	✗	✗	GELU	576	**584**	568		
✗	✓	✓	✗	✗	ReLU	480	**512**	248		
✗	✓	✗	✓	✓	GELU	688	**696**	680		
✗	✓	✗	✓	✓	ReLU	672	**688**	672		
✗	✓	✗	✓	✗	GELU	648	648	**688**		
✗	✓	✗	✓	✗	ReLU	**672**	**672**	**672**		
✗	✓	✗	✗	✓	GELU	536	560	**576**		
✗	✓	✗	✗	✓	ReLU	**552**	544	544		⋆
✗	✓	✗	✗	✗	GELU	536	**544**	536		
✗	✓	✗	✗	✗	ReLU	544	**552**	504		
✗	✗	✓	N/A	N/A	N/A	240	248	**256**		
✗	✗	✗	N/A	N/A	N/A	**64**	**64**	**64**		

Table 5: Maximum facts memorised per architectural configuration ($d_{\text{residual}} = 16$, $d_{\text{ff}} = 16$). Columns Dup, 3att, and Orig correspond to `experiment_log_3att_duplicate`, `experiment_log_3att`, and `experiment_log`, respectively. E1 corresponds to `E1/experiment_log` (attention-only configurations). The highest value in each row is shown in **bold**, outliers are highlighted with a box, and the ⋆ symbol indicates that this architectural configuration was chosen for scaling experiment in Section **??**.

						Max Facts			
Attention	**FF**	**Norms**	**Residual**	**Bias**	**Activation**	**3att_L**	**Dup**	**3att**	**Orig**
✓	✓	✓	✓	✓	GELU		800	744	**832**
✓	✓	✓	✓	✓	ReLU		**768**	**768**	728
✓	✓	✓	✓	✗	GELU	**832**	**832**	648	600
✓	✓	✓	✓	✗	ReLU	**744**	736	736	520
✓	✓	✓	✗	✓	GELU		**816**	800	600
✓	✓	✓	✗	✓	ReLU		**720**	712	448
✓	✓	✓	✗	✗	GELU	**768**	696	632	576
✓	✓	✓	✗	✗	ReLU	360	304	**408**	320
✓	✓	✗	✓	✓	GELU	**584**	544	432	264
✓	✓	✗	✓	✓	ReLU	240	416	**432**	120
✓	✓	✗	✓	✗	GELU	344	**592**	544	248
✓	✓	✗	✓	✗	ReLU	**584**	472	232	224
✓	✓	✗	✗	✓	GELU	464	408	336	**512**
✓	✓	✗	✗	✓	ReLU	312	248	**424**	320
✓	✓	✗	✗	✗	GELU	**472**	312	392	176
✓	✓	✗	✗	✗	ReLU	368	**432**	392	368
✓	✗	✓	N/A	N/A	N/A		144	152	**168**
✓	✗	✗	N/A	N/A	N/A		112	**144**	112
✗	✓	✓	✓	✓	GELU	**1016**	**1016**	**1016**	**1016**
✗	✓	✓	✓	✓	ReLU	**1016**	**1016**	**1016**	**1016**
✗	✓	✓	✓	✗	GELU	**1016**	**1016**	**1016**	1000
✗	✓	✓	✓	✗	ReLU	**1016**	**1016**	**1016**	960
✗	✓	✓	✗	✓	GELU	**1016**	**1016**	**1016**	**1016**
✗	✓	✓	✗	✓	ReLU	**1008**	1000	1000	504
✗	✓	✓	✗	✗	GELU	**896**	864	840	864
✗	✓	✓	✗	✗	ReLU	408	464	**544**	272
✗	✓	✗	✓	✓	GELU	960	**1000**	960	960
✗	✓	✗	✓	✓	ReLU	**960**	**960**	936	**960**
✗	✓	✗	✓	✗	GELU	**1016**	960	960	992
✗	✓	✗	✓	✗	ReLU	**960**	936	944	944
✗	✓	✗	✗	✓	GELU	**864**	800	808	832
✗	✓	✗	✗	✓	ReLU	800	744	**808**	624
✗	✓	✗	✗	✗	GELU	**744**	**744**	688	680
✗	✓	✗	✗	✗	ReLU	**648**	600	**648**	400
✗	✗	✓	N/A	N/A	N/A		**552**	520	544
✗	✗	✗	N/A	N/A	N/A		**216**	**216**	**216**

Table 6: Maximum facts memorised per architectural configuration using Cross-Entropy loss ($d_{\text{residual}} = 16$, $d_{\text{ff}} = 16$). Columns CE_Long, CE_Dup, CE_3att, and CE_Orig correspond to `experiment_log_CE_3att_long_duplicate`, `experiment_log_CE_3att_duplicate`, `experiment_log_CE_3att`, and `experiment_log_CE`, respectively. Empty cells indicate configurations not covered by that run. The highest value in each row is shown in **bold** and outliers are highlighted with a box.

Data and model dimension parameters	Small	Medium	Large	XL
`input_vocab_size`	32	64	128	256
`output_vocab_size`	16	32	64	128
`d_residual`	16	32	64	128
`d_ff`	16	32	64	128

Table 7: Data and model dimensions across different experiment scales.

Parameter	Full Model	Reduced Model
Architectural toggles		
`attention`	✓	✗
`ff`	✓	✓
`bias`	✓	✓
`norms`	✓	✗
`ff_residual`	✓	✗
`ff_activation_type`	GELU	ReLU

Table 8: The two model architectures used in the scaling experiment.